What is Machine Learning?

Machine Learning is a subset of AI where machines **learn patterns from data** and make predictions or decisions without being explicitly programmed for every rule.

Instead of writing manual if-else rules, you feed data → the algorithm finds the pattern - the model generalizes to new unseen data.

4 Types of Machine Learning

a) Supervised Learning

Data is **labeled** — the algorithm knows both input and correct output. It learns the mapping: input - output.

- Examples: Linear Regression, Logistic Regression, Random Forest, Decision Tree
- Two sub-types:
  - **Regression** → continuous output (e.g., predicting salary)
  - **Classification** → discrete output (e.g., 0, 1, 2, 3 — spam or not spam)

b) Unsupervised Learning

Data is **not labeled**. The algorithm finds hidden structure or groupings on its own.

- Example: Clustering - K-Means (groups similar data points together)

c) Semi-Supervised Learning

Label **half the data**, leave the rest unlabeled. The model learns from labeled data and infers patterns in the unlabeled portion.

d) Reinforcement Learning

The model (called an **agent**) tries actions, observes results, and learns by **trial and error**.

- Rewarded for correct behavior, penalized for wrong
- Pattern: observe → act → reward/penalty → repeat

Types of Regression

Simple Linear Regression

One input (x), one output (y). As x increases, y also increases — used for **continuous value prediction**.

Example from class: Years of Experience → Salary (30 data points)

```
y  =  mx + c          (Best Fit Line)
```

- m = slope (how much y changes per unit of x)
- c = intercept (value of y when x = 0)

Logistic Regression

Despite the name, used for **classification**. Output is categorical or discrete.

- Binary → 0 or 1 (e.g., spam/not spam, pass/fail, yes/no)
- Multi-class → 0, 1, 2, 3… (e.g., type of flower, digit recognition)

Multilinear Regression

Multiple independent features predicting one output.

```
y  =  β0 + β1·x1 + β2·x2 + ··· + βn·xn
```

- Key rule: no relationship between the input features (no multicollinearity)

Note: Use VIF (Variance Inflation Factor) to check for multicollinearity. VIF > 5 is a concern.

Polynomial Regression

When the relationship between x and y is **non-linear (curved)**, we raise x to higher powers.

```
y  =  β0 + β1·x + β2·x² + β3·x³ + ···
```

Note: Higher degree = fits training data better, but increases risk of overfitting.

Residuals, Cost Function & Gradient Descent

Residual

The difference between the **actual value** and what the model **predicted**. It is the vertical distance from each data point to the best fit line.

```
Residual (eᵢ)  =  yᵢ (actual)  −  ŷᵢ (predicted)
```

Cost Function (Loss Function)

Measures the **total error** of the model across all data points. The model's goal is to **minimize** this value.

Mean Squared Error (MSE)

```
MSE  =  (1/n) × Σ (yᵢ − ŷᵢ)²
```

Root Mean Squared Error (RMSE)

```
RMSE  =  √MSE          ←  same unit as y, more interpretable
```

Mean Absolute Error (MAE)

```
MAE  =  (1/n) × Σ |yᵢ − ŷᵢ|    ←  less sensitive to outliers
```

Gradient Descent

An optimization algorithm that iteratively adjusts the model's parameters (slope m, intercept c) to **minimize the cost function**.

Think of it as rolling a ball downhill — it keeps moving in the direction that reduces error until it finds the lowest point (minimum cost).

```
β  :=  β  −  α × ∂(Cost)/∂β
```

- α = learning rate (how big a step to take each iteration)
- Too large α → overshoots the minimum
- Too small α → takes too long to converge

05 MODEL EVALUATION

R^2 and Adjusted R^2

R^2 (R-squared)

Tells how well the model **explains the variance** in the data. Range: 0 to 1 (1 = perfect fit, 0 = no better than a flat line).

```
R²  =  1  −  (SS_residual / SS_total)
```

- R^2 = 0.95 means the model explains 95% of the variance in salary
- **Problem:** R^2 can increase or stay the same even when you add a useless/irrelevant feature — it never penalizes.

Adjusted R^2

Solves the R^2 problem - **penalizes adding unnecessary features**. Only increases if the new feature is genuinely useful.

```
Adj R²  =  1  −  [(1−R²)(n−1) / (n−k−1)]
```

- n = number of samples
- k = number of features
- Always prefer Adjusted R^2 when comparing models with different numbers of features

Bias, Variance & Tradeoff

Total Error = Bias² + Variance + Irreducible Noise

High Bias - Underfitting

Model is **too simple**. Leans toward missing the real pattern. High error on both training and test data.

- Cause: not enough features, model too rigid
- Fix: use a more complex model, add more relevant features

High Variance - Overfitting

Model is **too complex**. Memorizes the training noise. Very low training error but high test error.

- Cause: too many features, polynomial degree too high
- Fix: reduce complexity, use regularization (Ridge / Lasso), get more data

The Sweet Spot

The Tradeoff

You can't have zero bias AND zero variance at the same time. The goal is to find a **sweet spot**, Bias-Variance Tradeoff.

Summary table

Condition	Train Error	Test Error	Diagnosis
High Bias	High	High	Underfitting
High Variance	Low	High	Overfitting
Balanced (goal)	Low	Low	Good fit

class notes

- Bias = model leans toward more data / prior assumptions — misses the real pattern
- Variance = spread of data the model tries to capture
- Variance too low → underfit model (too rigid)
- Variance too high → overfit model (too flexible)
- Bias–Variance Tradeoff: as you reduce bias, variance tends to increase and vice versa